

LESSON - 12



SIKHOGE KYA???

INTRODUCTION & KEY POINTS

- INTRODUCTION
- OVERVIEW OF DATABASES
- APPLICATIONS & JOBS
- SOFTWARE INSTALLATION -
- MYSQL SERVER AND MYSQL WORK BENCH WITH DATABASE SETUP
- TABLES CREATION & FIRST QUERY

SQL STATEMENT FUNDAMENTALS

- SELECT * FROM
- SELECT DISTINCT
- COUNT, ALIAS AS
- SELECT WHERE
- LIMIT AND ORDER BY
- OPERATORS AND FILTERING
- BETWEEN, IN, LIKE AND ILIKE
- CASE IF ELSE
- COALESCE, NULLIF



DATA MANIPULATION (DML)

- INSERT
- UPDATE
- DELETE
- COPY RENAME STRUCTURE
- CHECK CONSTRAINT

DATA DEFINITION LANGUAGE (DDL)

- DATA TYPES & ROWS COLUMNS
- PRIMARY FOREIGN & OTHER KEYS
- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- CONSTRAINTS

JOINS & SUB QUERY

- INNER JOIN
- LEFT JOIN (LEFT OUTER JOIN)
- RIGHT JOIN (RIGHT OUTER JOIN)
- FULL JOIN (FULL OUTER JOIN)
- CROSS JOIN
- SELF JOIN
- -----
- UNION, UNION ALL, MINUS, INTERSECT
- STRING FUNCTIONS AND OPERATORS
- SUBQUERY - IMPORTANT
- DATE & TIME FUNCTIONS

GROUP BY & HAVING

- INTRODUCTION TO GROUP BY
- AGGREGATION FUNCTIONS
- GROUP BY - CHALLENGE
- HAVING
- HAVING - CHALLENGE TASKS

ADVANCE SQL

- VIEWS / TEMP TABLES & INDEXES
- FUNCTIONS & TRIGGERS
- STORE PROCEDURES
- TRANSACTIONS
- WINDOW FUNCTIONS
- QUERY ANALYSIS TECHNIQUES

JOINS



SQL JOIN clause is used to query and access data from multiple tables by establishing logical relationships between them. It can access data from multiple tables simultaneously using common key values shared across different tables.

We can use SQL JOIN with multiple tables. It can also be paired with other clauses, the most popular use will be using JOIN with WHERE clause to filter data retrieval.

INNER JOIN / JOIN

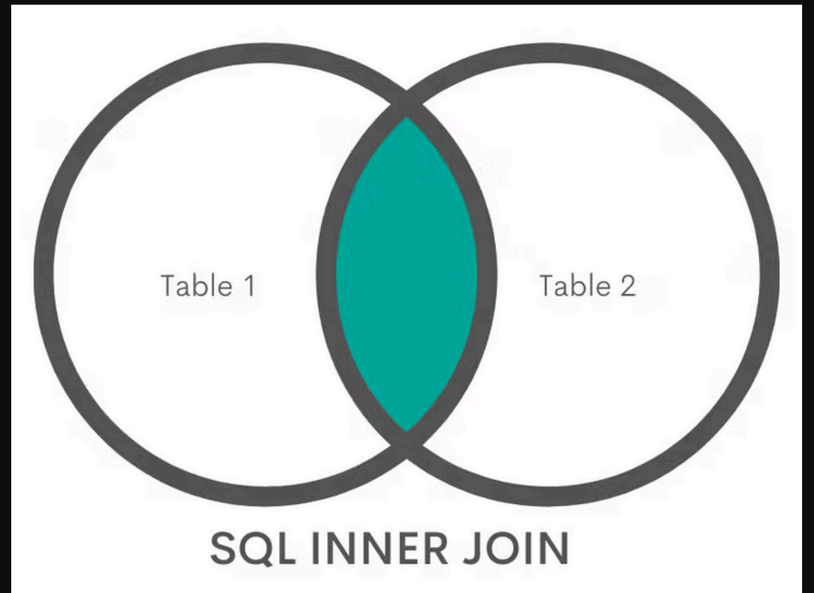
By default, JOIN is equivalent to INNER JOIN and retrieves records that have matching values in both tables.

Syntax:

```
SELECT column1, column2, ... FROM table1 JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT students.name, courses.course_name FROM students JOIN courses ON students.course_id = courses.course_id;
```



LEFT JOIN

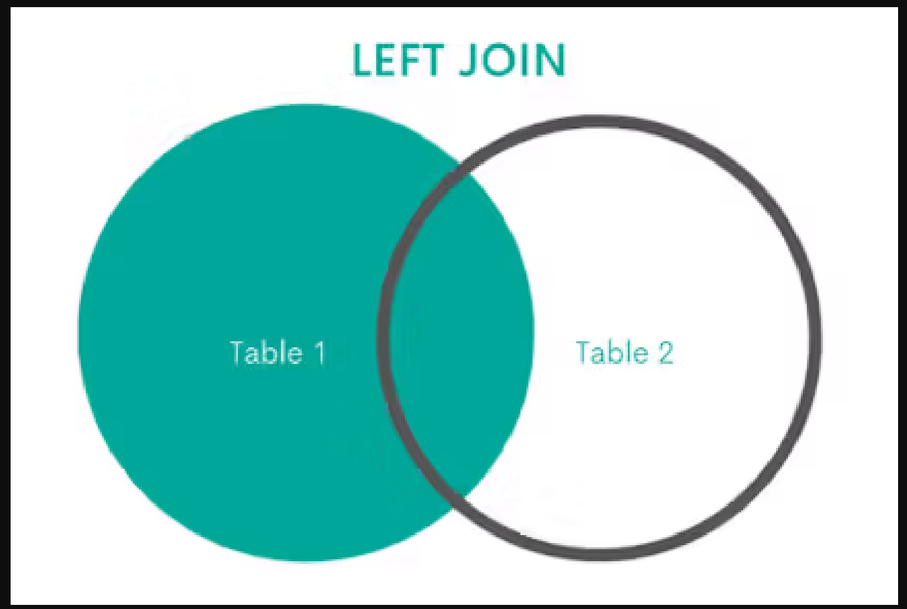
Returns all records from the left table and the matched records from the right table. If no match is found, NULL values are returned for columns from the right table.

Syntax:

```
SELECT column1, column2, ... FROM table1 LEFT JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT customers.customer_name, orders.order_date FROM customers LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```



RIGHT JOIN

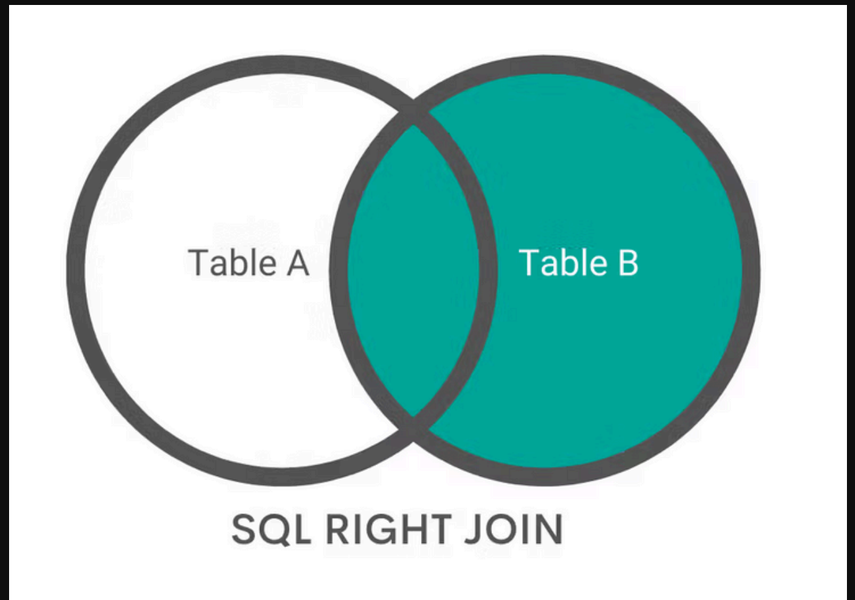
Returns all records from the right table and the matched records from the left table. If no match is found, NULL values are returned for columns from the left table.

Syntax:

```
SELECT column1, column2, ... FROM table1 RIGHT JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT employees.name, projects.project_name FROM employees RIGHT JOIN projects ON employees.project_id = projects.project_id;
```



FULL OUTER JOIN

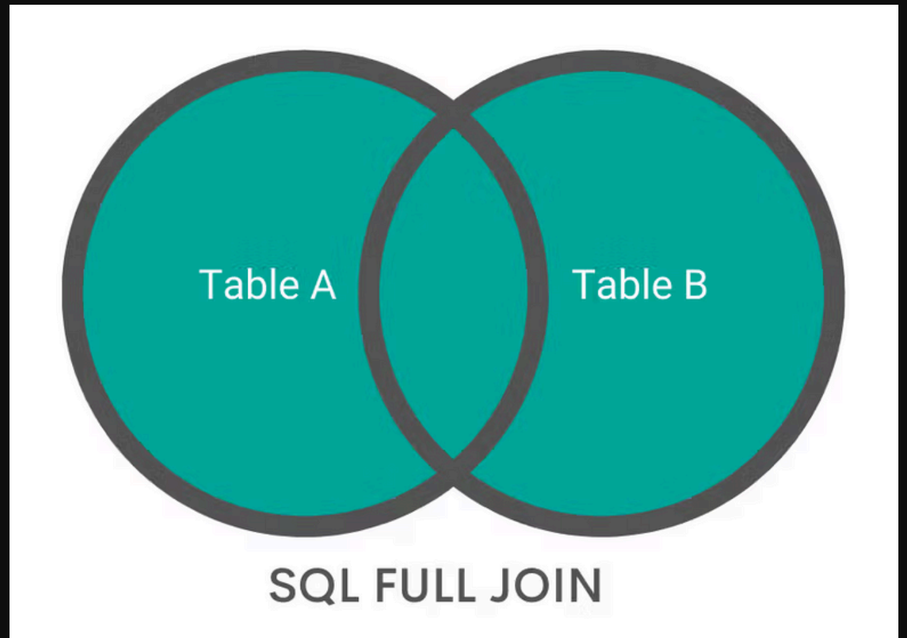
One of the misunderstood types of JOINs in SQL. It takes columns and rows from both tables, even those that do not match. When two tables overlap, FULL OUTER JOIN output values from both columns.

Syntax:

```
SELECT column1, column2, ... FROM table1 FULL JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT a.name, b.salary FROM employees a FULL JOIN salaries b ON a.employee_id = b.employee_id;
```



CROSS JOIN

CROSS JOIN generates all possible combinations between all the rows in two tables. They are typically used to get the number of possible combinations of rows.

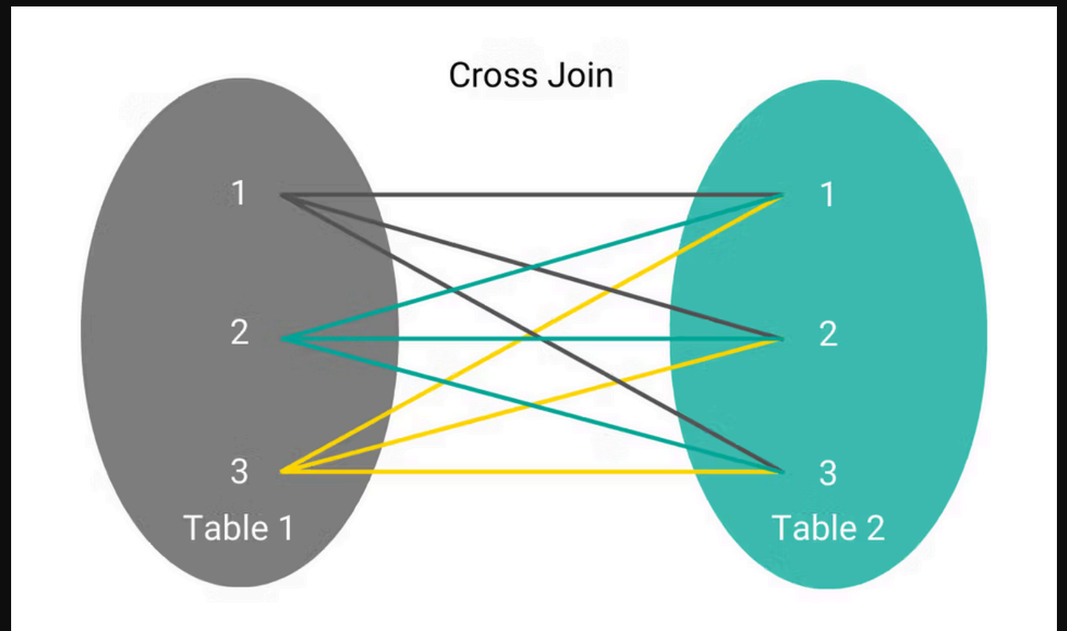
It does not require a condition like ON or USING as in INNER JOIN or LEFT JOIN.

Syntax:

```
SELECT column1, column2, ... FROM table1 CROSS JOIN table2;
```

Example:

```
SELECT products.product_name, categories.category_name FROM products CROSS JOIN categories;
```



SELF JOIN

Joins a table with itself. Useful for hierarchical data or comparing rows within the same table.

Syntax:

```
SELECT a.column1, b.column2 FROM table_name a, table_name b WHERE condition;
```

Example:

```
SELECT e1.name AS Employee, e2.name AS Manager FROM employees e1 INNER JOIN employees e2 ON e1.manager_id =
e2.employee_id;
```

